Day 18 – Project Report

Distributed Memory Grid for Real-Time LLM Inference

1. Project Overview

LLM MemoryGrid is a prototype application that provides an in-memory cache for prompt processing. It uses FastAPI for the backend APIs and React with Vite for the frontend. The current inference implementation is a stub, so the project can be run and tested without a production LLM provider.

2. System Architecture

- FastAPI backend: app/main.py handles API requests, validation, PDF processing, and request timing logs.
- Memory grid: app/grid/ contains memory nodes, deterministic hash-based routing, TTL expiration, and LRU eviction. The router maps normalized keys to nodes using a hash modulo the configured node count.
- Worker: worker/ runs prompt processing in a separate process, with a five-second default timeout and failure handling.
- Frontend: frontend/ contains the React and Vite user interface.

3. API Endpoints

Method | Endpoint | Purpose
GET | /health | Check API health.
POST | /api/prompt | Submit a prompt; check the cache and run worker inference on a miss.
POST | /api/pdf/today | Upload a PDF, extract its text, and find sentences matching a date.
GET | /api/grid/config | View grid configuration.
GET | /api/grid/health | Check grid health and cached-entry totals.
GET | /api/grid/stats | View per-node entries and cache hit/miss/hit-rate metrics.
POST | /api/grid/clear | Clear cached entries and reset cache metrics.

4. Key Features

- Deterministic hash-based routing across memory nodes.
- In-memory prompt caching with configurable default TTL.
- PDF text cached with a 60-second TTL.
- LRU eviction when a node reaches capacity.
- Cache hit and miss counters, with a calculated hit rate.
- Worker timeout and failure isolation using a separate process.
- Request timing middleware that logs method, path, status, and elapsed time.
- PDF text extraction and date matching.

5. Testing

Run the complete test suite:

python -m pytest tests/ -v

Run the Day 17 integration tests:

python -m pytest tests/test_day17_integration.py -v

The Day 18 run collected 93 tests and exposed a stale health-status assertion. On the Day 19 clean main baseline, the full suite passed: 94 passed, 0 failed, with 3 deprecation warnings. The fresh verbose run is saved as test_results_day19.txt in the Day 19 regression contribution.

6. Development Environment

- Python
- FastAPI and Pydantic
- React and Vite
- Pytest
- Uvicorn

7. Current Development Status

Backend API functionality, cache management, worker processing, PDF handling, frontend integration, and automated tests are implemented. The full suite is green on the Day 19 baseline. The demo portal now includes PDF date lookup and cache/grid statistics alongside prompt submission.

Further work includes reviewing hash-routing behavior under node changes, testing performance, and integrating a production LLM inference provider.

8. Conclusion

LLM MemoryGrid demonstrates an architecture for reducing repeated prompt-processing work through caching and memory-node routing. It provides a foundation for further development and performance evaluation.

9. Team Contributions

- Member 1 (Lead): Architecture, FastAPI application endpoints, reviews, and this project report.
- Member 2: Memory grid design and implementation, including deterministic hash routing, TTL expiration, and per-node LRU eviction.
- Member 3: PDF text extraction with pypdf, date matching across supported formats, and malformed/encrypted PDF handling.
- Member 4: Isolated worker processing, timeout and failure handling, and worker/API tests.
- Member 5: React frontend prompt portal, PDF upload, and cache/grid statistics dashboard.

10. Day 19 Results

- Repeated prompt test verified a cache miss followed by a cache hit.
- Memory-node tests verified least-recently-used eviction while staying within configured capacity.
- Worker failure tests verified a clean HTTP 500 response; timeout tests verified a bounded failure response rather than a hanging request.
- PDF endpoint tests verified uploaded PDF extraction and matching date results.
- Full regression command python -m pytest tests/ -v: 94 passed, 0 failed, 3 warnings. See test_results_day19.txt for the captured run.